SOFTWARE ARCHITECTURE

Microservices and BAAS

BEYOND BOUNDARIES

HOA SEN UNIVERSITY
WORLD CLASS EDUCATION

QS STARS RATING SYSTEM

ACBSP ACCREDITED

AUN-QA

MOET ACCREDITED
KIỂM ĐỊNH CHẤT LƯỢNG GIÁO DỤC QUỐC GIA

Content

- Microservices
  - Microservices definition
  - Distinction from SOA and monolith applications
  - Architectural elements
  - Architectural motivations
- Backend As A Service

Microservices architecture definition

- Microservices architecture is a collection of microservices.
- A microservice can be defined as follows:
  - The smallest service that does **only one thing**, that is, Single Responsibility Principle (SRP)
  - It's an independent piece of code and independently manageable without dangling dependencies
  - It's the owner of its own data; no sharing except via services
- It is an architectural approach to develop an application (or a system) as a set of small services, where each service works independently in its own process space and communicates using lightweight mechanisms.
- The services are naturally built around business capabilities just like in SOA and are independently deployable components as described by bounded contexts in DDD (Domain Driven Design).

Microservices and SOA

- The advent of microservices has disrupted the existence and architecting for SOA.
- Although microservices and SOA are similar things, as both are service oriented, there are some subtle technical differences in architecture and their implementation.
- With the high usage of cloud-based applications, DevOps processes, and agile methodologies, it's becoming more common, and rather natural, to follow the microservices approach.

Microservices and SOA

- In SOA, all organizations tend to follow a certain theme, standards, and programming languages with conventions **uniformly across pieces of code**.
- On the other hand, in microservices, each microservice package is **independent**--it can use its own programming language, its own configuration, and data (SQL, NoSQL), which is common only across its team.
- Two microservices packages can be implemented entirely differently using different implementation technologies while having a common way of exposing and consuming the service interfaces.

Microservices and SOA

- In SOA, heavy focus is on **centralization**, and therefore, inclusion of a certain Enterprise Service Bus (ESB) is inevitable.
- In microservices, the focus is on **decentralization** in the form of completely independent components designed for failure situations independently.
- **Microservices are sometimes even referred to as an ESB anti-pattern**.
- In SOA, centralized governance often inhibits change, while in microservices, each service package is its own owner, that is, completely autonomous.

Microservices & Monolithic Applications (1/2)

- Monolithic applications are applications that are deployed as a single executable unit or a package (for example, deployment archive). Enterprise applications are often built in three main parts, which are as follows:
  - Frontend
  - Middle-tier / server-side-code
  - Backend (database)

Microservices & Monolithic Applications (2/2)

- The server-side code (internally designed as a layered architecture but) is a single logical executable unit, and often, a single process.
- Any change to the application needs to build and deploy the whole server-side part to the production server.
- Monolithic applications can **scale horizontally** by replicating the whole server-side application instance into multiple servers or VMs. Therefore, scaling requires scaling of the entire application instead of the part of application that needs extra resources.
- Microservices architecture divides applications into **several autonomous microservices**. Each microservice is a self-contained package of all code along with its data without unmanaged external dependencies.:

Web API and web services

- Microservice is often referred to as a remotely callable API, but it is much more than the older network-callable API or remote process execution standards such as Remote Method Invocation (RMI), Common Object Request Broker Architecture (CORBA), or Distributed Component Object Model (DCOM) - callable API standards.
- Web API is, however, most common in the Microsoft world, where Web APIs (server-side) are REST-based services that use HTTP verbs to define actions on the business resources (data). This kind of service typically uses JSON or XML (or sometimes in binary) as the request and response format.
- Web services are services which are traditionally SOAP (simple object access protocol) based services whose request and responses are in XML, or sometimes, in binary. A web service is typically exposed by a standard WSDL (web services description language) as an interface definition language, which is used to create the proxies at the web service caller end.

Characteristics of a microservices architecture

- Microservices architecture is more of a concept than a defined architecture with a specific set of requirements.
- It's a better way to achieve the software, which is built to scale reliably and responsively.

Characteristics of a microservices architecture

- **Componentization via services**: A component is a unit of software that is independently replaceable and upgradeable. The advantage of using services as components (rather than libraries) in a whole system, is that services are independently deployable.
- **Organized around business capabilities**: Services directly reflect the business capabilities and achieve the related business processes in the respective business domains and bounded-contexts.
- **Products not Projects**: It is a notion that the development team should own the product for its lifetime. The product-based mentality makes closer ties to business capabilities. It brings the developer nearer to the business users or consumers and helps him/her see how the software behaves in production.

Advantages of microservices

- **Improved fault isolation**: Larger applications can remain mostly unaffected by the failure of a single module.
- **Eliminate vendor or technology lock-in**: Microservices provide the flexibility to try out a new technology stack on an individual service as needed. There won't be as many dependency concerns and rolling back changes becomes much easier. With less code in play, there is more flexibility.
- **Ease of understanding**: With added simplicity, developers can better understand the functionality of a service.
- **Smaller and faster deployments:** Smaller codebases and scope = quicker deployments, which also allow you to start to explore the benefits of Continuous Deployment.
- **Scalability**: Since your services are separate, you can more easily scale the most needed ones at the appropriate times, as opposed to the whole application. When done correctly, this can impact cost savings.

Disadvantages of microservices

- **Communication between services is complex**: Since everything is now an independent service, you must carefully handle requests traveling between your modules.
- **More services equals more resources:** Multiple databases and transaction management can be painful.
- **Global testing is difficult**: Testing a microservices-based application can be cumbersome.
- **Debugging problems can be harder**: Each service has its own set of logs to go through. Log, logs, and more logs.
- **Deployment challengers**: The product may need coordination among multiple services, which may not be as straightforward as deploying a WAR in a container.
- **Large vs small product companies**: Microservices are great for large companies but can be slower to implement and too complicated for small companies who need to create and iterate quickly, and don't want to get bogged down in complex orchestration.

Backend As A Service (BAAS)

- Cloud computing
  - IAAS
  - PAAS
  - SAAS

BAAS

- With cloud computing and the advent of microservices, BaaS started to gain momentum as well
- BaaS systems are those that provide infrastructure software services, and these **include storage as a service, database as a service, messaging, and notifications as a service.**
- With serverless architecture, applications usually use a combination of BaaS services, which are often provided by the vendor cloud echo system such as Google's Firebase and Amazon DynamoDB.
- Other third-party BaaS services may include authentication services (like Auth0, AWS Cognito, and Azure Active Directory).

BAAS Example

- AWS Lambda and Azure Functions.
- There could also be a single service that attempts to provide a complete set of capabilities (for example, data stores, messaging servers, and so on) around BaaS, which is inclusive of number of BaaS services.
- Examples of such a service provider include Parse backend or Firebase, to read data using either the ID of a specific Parse object

What is Backend As A Service?

- Backend as a service (BaaS) is a third-party service that lets you develop an app or a website without bothering with its backend.
- When using this service, you leave time-consuming and expensive backend tasks to a BaaS provider while channeling your efforts into the frontend.
- BaaS grants you access to a bundled infrastructure with tailor-made APIs and SDKs behind it.
- It saves you the pain of powering your development, managing user permissions, setting up cloud storage, and configuring user authentication.

When should you use Backend As A Service?

- Game apps, whether on Android or iOS
- SPAs (single-page applications)
- Websites
- Chatbots
- E-commerce apps
- Data management apps
- And more

BaaS architecture and key components

Modules

API

Data channels

Modules

- Content-types builder
  - Developers can manage entries and fields to frame how the data is served over the backend to users
- Content-types data
  - You can filter queries with GraphQL or APIs and perform CRUD (create, read, update, and delete) operations.
  - Some BaaS providers also offer a business logic programming option within this module to help you control app data exchange processes.
- User models.
  - A BaaS strongbox of authentication processes and permissions. You turn to it to oversee how users log into your web or mobile app, manage what information they can access, and enable data sharing with third parties.

API

- An API is the second layer of the BaaS architecture cake.
- Today, BaaS platforms bet on REST APIs that help developers stay nimble with backend operations.
- REST APIs simplify the management and configuration of BaaS architecture modules, including authentication and database changes.
- Another upside is that they make BaaS valuable for any app you can think of — from games to chatbot applications.

Data channels

- Data channels are the highways that data travel on to reach the ultimate backend destination, which is a REST API.
- Once the data from your app channels are fed into a BaaS API, it initiates interactions with databases, servers, and modules.
- Specifically, these channels are like the circulatory system of a BaaS platform architecture.

What features of BAAS can you get?

- Data storage
  - A BaaS platform provides unlimited backend storage for your app or website data.
  - Your developers only need to set it up and watch the data flow consistently to where it can be securely kept and served upon request.
- Data manipulation using an API
  - BaaS and APIs go together, even when it comes to data manipulation.
  - This feature is vital for organizing, mapping, sprucing up, or otherwise modifying data to be further fed into the backend or delivered to users.

What features of BAAS can you get? (1/2)

- User modeling
  - User modeling is a colossal chunk of backend functionality, allowing developers to manage roles, permissions, and groups.
  - With this feature, you can control authorization scopes and user access to any part of your product.
  - If you're developing a customer portal, user modeling is your way to create a walled environment that authorized users can flock to and interact with their data.

What features of BAAS can you get? (2/2)

- File storage
  - BaaS platforms take good care of file storage for mobile and web apps using specifically built modules.
  - You don’t even need to fret about backups.
  - Your BaaS provider is permanently on guard against system failures so that your files are safe and sound, no matter what

Key takeaways of BAAS

- It saves your time, which you can use to spruce up your frontend
- It expedites MVP development and prototyping
- It reduced backend workload drives cost-efficiency
- It includes many features and tools you wouldn’t otherwise have free access to
- BaaS vendors step up security for your web or app project

Q&A

This Photo by Unknown Author is licensed under CC BY-NC-ND